

SilkSphere Platform

AI-Powered Sericulture Management System

Project Report | Version 1.0.0 | April 2026

GitHub: <https://github.com/Himanshu13chib/SilkSphere>

SilkSphere Platform ? Project Report

Project Title: SilkSphere ? AI-Powered Sericulture Management Platform

Technology Stack: React.js, Vite, FastAPI, TensorFlow/Keras, Python

Version: 1.0.0

GitHub: <https://github.com/Himanshu13chib/SilkSphere>

1. Project Overview

SilkSphere is a full-stack web application designed to modernize and digitize the sericulture (silk farming) industry in India. It bridges the gap between silkworm farmers and industry buyers through a smart, data-driven platform that combines real-time IoT sensor monitoring, AI-powered disease detection, lifecycle tracking, and a digital marketplace.

The platform serves two types of users:

- Farmers ? who raise silkworms and sell cocoon batches
- Industry Buyers ? who purchase graded, verified batches

2. Problem Statement

The Indian sericulture industry faces several challenges:

- Silkworm diseases (especially Grasserie) cause massive crop losses with no early detection system
- Farmers lack tools to monitor environmental conditions in real time
- No transparent marketplace exists for graded silk batches
- Manual quality grading is inconsistent and unreliable
- Buyers have no visibility into the health history of batches they purchase

SilkSphere addresses all of these with a unified digital platform.

3. System Architecture

```

????????????????????????????????????????????????????????????
?          FRONTEND (React.js)          ?
?  ?????????????? ?????????????? ?????????????? ?????????????? ?
?  ?  Auth      ? ? Farmer  ? ? Buyer  ? ?Shared  ? ?
?  ?  Pages    ? ? Pages   ? ? Pages   ? ? Pages   ? ?
?  ?????????????? ?????????????? ?????????????? ?????????????? ?
?          AppContext (Global State)      ?
????????????????????????????????????????????????????????????
?          ? HTTP (REST API)              ?
????????????????????????????????????????????????????????????
?          BACKEND (FastAPI / Python)      ?
?  ?????????????? ?????????????? ?????????????? ?????????????? ?
?  ?          /predict (POST)              ?  ?
?  ?  Image ? MobileNetV2 Model ? Prediction  ?  ?
?  ?????????????? ?????????????? ?????????????? ?????????????? ?
?  Rate Limiting · CORS · Input Validation      ?
????????????????????????????????????????????????????????????
?
????????????????????????????????????????????????????????????
?          AI MODEL (silkworm_disease_model.keras)  ?
?  Architecture: MobileNetV2 + Dense(128) + Dense(1) ?
?  Input: 224×224 RGB image                        ?
?  Output: sigmoid (0=Grasserie, 1=Healthy)         ?
????????????????????????????????????????????????????????????

```

4. Technology Stack

Frontend

Technology	Version	Purpose
React.js	19.2.4	UI framework
Vite	8.0.4	Build tool & dev server
Recharts	3.8.1	Charts and data visualization
Lucide React	1.7.0	Icon library
CSS (custom)	?	Styling (no Tailwind/Bootstrap)

Backend

Technology	Version	Purpose
Python	3.11.9	Runtime

FastAPI	0.136.0	REST API framework
TensorFlow	2.21.0	Deep learning inference
Keras	3.14.0	Model loading
Pillow	12.x	Image preprocessing
Uvicorn	0.44.0	ASGI server
SlowAPI	0.1.9	Rate limiting
Firebase Admin	7.4.0	Authentication (optional)

AI Model

Property	Value
Base Architecture	MobileNetV2
Input Shape	(224, 224, 3)
Output	Sigmoid (binary classification)
Classes	Healthy / Grasserie (infected)
Saved Format	.keras (Keras 3.14)
File Size	~11 MB

5. Frontend Architecture

5.1 Project Structure

```
SilkSphere/src/
??? App.jsx           # Root component
??? main.jsx          # Entry point
??? context/
?   ??? AppContext.jsx # Global state management
??? components/
?   ??? layout/
?   ?   ??? AppShell.jsx # Main layout wrapper
?   ?   ??? Sidebar.jsx  # Navigation sidebar
?   ?   ??? Topbar.jsx   # Top navigation bar
?   ??? ui/
?   ?   ??? SilkLogo.jsx  # Brand logo component
?   ?   ??? KiroOrb.jsx   # AI assistant orb
?   ?   ??? SvgGauge.jsx  # Gauge chart component
?   ?   ??? ToastContainer.jsx # Notification toasts
?   ??? charts/
?       ??? GaugeChart.jsx # SVG arc gauge
??? pages/
?   ??? auth/
?   ?   ??? AuthRouter.jsx # Auth routing
?   ?   ??? LoginPage.jsx  # Login screen
?   ?   ??? RegisterPage.jsx # Registration screen
```

```
?   ??? farmer/
?   ?   ??? Dashboard.jsx      # Farmer dashboard
?   ?   ??? MyBatches.jsx      # Batch management
?   ?   ??? AIDiagnostics.jsx  # AI disease detection
?   ?   ??? LifeCycle.jsx      # Lifecycle tracker
?   ?   ??? Environment.jsx    # IoT sensor data
?   ?   ??? Analytics.jsx      # Charts & analytics
?   ?   ??? FarmerMarketplace.jsx # Farmer's market view
?   ?   ??? FarmerRouter.jsx   # Farmer page routing
?   ??? buyer/
?   ?   ??? BuyerMarketplace.jsx # Browse & buy batches
?   ?   ??? MyOrders.jsx       # Order history
?   ?   ??? BuyerRouter.jsx    # Buyer page routing
?   ??? shared/
?       ??? Settings.jsx       # User settings
?       ??? Wallet.jsx         # Digital wallet
??? styles/
    ??? global.css             # Global styles & CSS variables
    ??? auth.css               # Auth page styles
```

5.2 State Management

The app uses React Context API (AppContext) for global state. No Redux or external state library is used.

State managed globally:

- user ? logged-in user object (role, name, email)
- sensor ? live IoT sensor readings (temperature, humidity, CO?)
- batches ? all silkworm batches
- orders ? purchase orders
- cart ? buyer's shopping cart
- wallet ? balance and transaction history
- alerts ? system notifications
- lifecycleStages ? silkworm lifecycle data
- toasts ? UI notification messages

Session persistence: User sessions are stored in localStorage and restored on page reload.

5.3 Routing

The app uses conditional rendering (not React Router) for navigation:

- If no user ? show AuthRouter (Login / Register)
- If user with role farmer ? show FarmerRouter
- If user with role buyer ? show BuyerRouter

Page navigation within each role is handled by a page state variable passed through the AppShell.

6. Pages & Features

6.1 Authentication

Login Page (LoginPage.jsx)

- Email + password login form
- SilkSphere logo panel on the left
- Role-based redirect after login
- Session stored in localStorage

Register Page (RegisterPage.jsx)

- Step 1: Choose role (Farmer / Industry Buyer)
- Step 2: Fill details (name, phone, state, email, password)
- Farmer fields: Farm name, phone, state
- Buyer fields: Company name, phone
- Form validation with trim checks

6.2 Farmer Pages

Dashboard (Dashboard.jsx)

- Live IoT sensor cards: Temperature, Humidity, CO₂, Light
- Environmental history chart (Recharts AreaChart)
- Active batch overview
- Quick navigation to AI Diagnostics
- Alerts banner for threshold breaches

My Batches (MyBatches.jsx)

- Table of all silkworm batches with ID, stage, scores, grade
- Register new batch modal
- Grade & list batch modal (A/B/C grading)
- AI Scan shortcut per batch
- Stats: total, active, ready to list, alerts

AI Diagnostics (AIDiagnostics.jsx)

- Live camera capture (getUserMedia API)
- Image upload support
- Sends image to FastAPI backend /predict endpoint
- Displays result: Healthy / Infected (Grasserie)
- Confidence percentage with progress bar
- Scan history (last 10 scans)
- Backend status indicator (Online / Demo Mode)
- Falls back to simulated results if backend is offline

Life Cycle (LifeCycle.jsx)

- Visual timeline of silkworm lifecycle stages
- Stages: Egg ? Instar 1-5 ? Spinning ? Cocoon
- Per-stage metrics: temperature, humidity, CO₂, mortality

- Event log per stage with timestamps
- Active stage highlighting

Environment (Environment.jsx)

- Real-time IoT sensor readings
- Historical trend charts
- Threshold alerts
- Sensor node status

Analytics (Analytics.jsx)

- Batch performance charts
- Environmental score trends
- AI health score history
- Grade distribution

Farmer Marketplace (FarmerMarketplace.jsx)

- View own listed batches
- Manage listings
- View incoming orders

6.3 Buyer Pages

Buyer Marketplace (BuyerMarketplace.jsx)

- Browse all listed batches from farmers
- Filter by grade (A/B/C), price, stage
- View batch details: env score, AI score, grade, price
- Add to cart functionality
- Place orders

My Orders (MyOrders.jsx)

- Order history with status (pending/confirmed)
- Batch details per order
- Order timeline

6.4 Shared Pages

Wallet (Wallet.jsx)

- Current balance display
- Add money (UPI, card, net banking)
- Withdraw money (bank transfer, UPI)
- Transaction history with type, method, date, amount

Settings (Settings.jsx)

- Profile information
- Account preferences

- Notification settings

7. AI Model ? Silkworm Disease Detection

7.1 Overview

The AI model detects Grasserie disease in silkworms from images. Grasserie (Nuclear Polyhedrosis Virus) is one of the most destructive silkworm diseases, causing significant crop losses.

7.2 Model Architecture

```
Input Layer (224 × 224 × 3)
    ?
MobileNetV2 Base (pretrained on ImageNet)
    - 154 layers of depthwise separable convolutions
    - Frozen weights (transfer learning)
    ?
Global Average Pooling 2D
    ?
Dense(128, activation='relu')
    ?
Dropout
    ?
Dense(1, activation='sigmoid')
    ?
Output: 0.0 = Grasserie (Infected) | 1.0 = Healthy
```

Why MobileNetV2?

- Lightweight and efficient ? suitable for deployment
- Pretrained on ImageNet ? strong feature extraction
- Depthwise separable convolutions reduce computation
- Proven performance on image classification tasks

7.3 Input Preprocessing

```
# MobileNetV2 specific preprocessing
image = image.resize((224, 224))
img_array = tf.keras.applications.mobilenet_v2.preprocess_input(
    np.array(image, dtype=np.float32)
)
# Scales pixel values from [0, 255] to [-1, 1]
```

7.4 Output Interpretation

```
prediction = model.predict(img_array)[0][0] # sigmoid value 0.0 to 1.0
```

```
if prediction > 0.5:
    class_name = "Healthy"
    confidence = prediction * 100
else:
    class_name = "Grasserie"
    confidence = (1.0 - prediction) * 100
```

7.5 Model File

- Format: .keras (Keras 3.14 format ? zip archive containing config.json + model.weights.h5)
- Size: ~11 MB
- Saved: April 18, 2026
- Location: backend/silkworm_disease_model.keras

8. Backend API

8.1 Overview

The backend is a FastAPI application that serves the AI model securely. The model is never exposed to the frontend ? all inference happens server-side.

8.2 Endpoints

GET /

Health check endpoint.

```
{
  "status": "Secure API is running",
  "model_loaded": true
}
```

POST /predict

Accepts an image file and returns disease prediction.

Request: multipart/form-data with field file (JPG/PNG, max 5MB)

Response:

```
{
  "class_name": "Healthy",
  "confidence": 0.9998,
  "timestamp": 1776531613.05
}
```

Validations:

- File type must be jpg, jpeg, or png
- File size must be under 5MB

- Rate limited to 30 requests/minute per IP

8.3 Security Features

1. Rate Limiting ? SlowAPI limits to 30 requests/minute per IP
2. CORS ? Configured to allow frontend origin
3. Input Validation ? File type and size checks
4. Model Protection ? Model weights never sent to client
5. Firebase Auth ? JWT token verification (optional, ready to enable)

8.4 Running the Backend

```
# Install dependencies
pip install -r requirements.txt

# Start server
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
```

9. Database & Storage

The current version uses localStorage (browser storage) for:

- User accounts and sessions
- Batch data
- Orders
- Wallet transactions

This is suitable for demo/prototype. For production, a real database (Firebase Firestore / PostgreSQL) would be used. Firebase Admin SDK is already integrated in the backend for future use.

10. Key Algorithms & Logic

10.1 Batch Grading

Batches are graded A/B/C based on:

- Environmental Score (temperature, humidity, CO₂ compliance)
- AI Health Score (from disease detection scans)
- Manual farmer assessment

10.2 Sensor Simulation

```
function generateSensor() {
  return {
    temperature: +(24 + Math.random() * 4).toFixed(1), // 24?28°C
    humidity: +(70 + Math.random() * 15).toFixed(1),    // 70?85%
```

```
    co2: Math.floor(800 + Math.random() * 400),          // 800?1200 ppm
  }
}
// Refreshes every 5 seconds
```

10.3 Session Management

```
// On login ? save to localStorage
localStorage.setItem('ss_session', JSON.stringify(userObject))

// On app load ? restore session
const session = localStorage.getItem('ss_session')
if (session) setUser(JSON.parse(session))

// On logout ? clear session
localStorage.removeItem('ss_session')
```

11. UI/UX Design

Design Principles

- Dark green and black color scheme reflecting sericulture/nature
- Sidebar navigation with black background
- Card-based layout for data display
- Responsive design for mobile and desktop
- Toast notifications for user feedback
- Loading states and animations

CSS Variables (Design Tokens)

```
--green: #2e7d32
--accent: #4caf50
--bg: #f8fdf8
--white: #ffffff
--text: #1a1a1a
--border: #e0e0e0
--red: #ef5350
--sidebar: #000000
```

Component Library

All UI components are custom-built (no Material UI, Ant Design, etc.):

- Cards, buttons, badges, pills
- Form inputs, selects, modals
- Progress bars, gauges
- Tables, charts

12. Deployment

Frontend ? Vercel

- Platform: Vercel (free tier)
- Build command: npm run build
- Output directory: dist
- vercel.json configured for SPA routing

Backend ? Railway (recommended)

- Platform: Railway / Render
- Start command: uvicorn backend.main:app --host 0.0.0.0 --port \$PORT
- Environment variable: VITE_BACKEND_URL set in Vercel

Environment Variables

```
VITE_BACKEND_URL=https://your-backend.railway.app
```

13. How to Run Locally

Prerequisites

- Node.js 18+
- Python 3.11+

Frontend

```
cd SilkSphere
npm install
npm run dev
# Opens at http://localhost:5173
```

Backend

```
cd backend
pip install -r requirements.txt
python -m uvicorn backend.main:app --host 127.0.0.1 --port 8000
# API at http://127.0.0.1:8000
```

Test Credentials

Role	Email	Password
Farmer	farmer@silk.com	demo123
Buyer	buyer@silk.com	demo123

14. Future Enhancements

1. Real IoT Integration ? Connect ESP32 + BME280 + MQ135 sensors via Firebase Realtime Database
2. Firebase Authentication ? Replace localStorage auth with Firebase Auth (JWT already wired in backend)
3. Multi-disease Detection ? Extend model to detect Flacherie, Muscardine, and other silkworm diseases
4. Blockchain Traceability ? Immutable batch history on blockchain for buyer trust
5. Mobile App ? React Native version for farmers in the field
6. Price Prediction ? ML model to predict optimal selling price based on grade and market trends
7. Government Integration ? Connect with Central Silk Board APIs for subsidy tracking

15. Team & Acknowledgements

- Frontend Development: React.js, Vite, custom CSS
- AI Model: MobileNetV2 transfer learning, TensorFlow/Keras
- Backend API: FastAPI, Python
- Platform: SilkSphere Technologies

This document covers the complete technical architecture, codebase, AI model, and deployment of the SilkSphere platform as of April 2026.